

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY code_lock_simple_fsm IS
5
6      PORT (
7          --In
8          reset : IN STD_LOGIC;
9          enter : IN STD_LOGIC;
10         code : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
11         clk : IN STD_LOGIC;
12         failed : IN STD_LOGIC;
13         --Out
14         lock : OUT STD_LOGIC;
15         lock0 : OUT STD_LOGIC;
16         err_event : OUT STD_LOGIC
17     );
18 END code_lock_simple_fsm;
19
20 ARCHITECTURE processes OF code_lock_simple_fsm IS
21     TYPE state IS (idle, Ev_code1, get_code2, Ev_code2, unlocked, perm_lock,
22     wrong_code);
23     SIGNAL present_state, next_state : state;
24
25     ----- Hardwirede koder til kodelåsen
26     SIGNAL code1 : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1100";
27     SIGNAL code2 : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1110";
28 BEGIN
29     ---STATE Registration-----
30     state_reg : PROCESS (clk)
31     BEGIN
32         IF rising_edge(clk) THEN
33             IF reset = '0' THEN
34                 present_state <= idle;
35             ELSE
36                 present_state <= next_state;
37             END IF;
38         END IF;
39     END PROCESS;
40
41     --- next state-----
42     nxt_state : PROCESS (present_state, code, code1, code2, enter)
43     BEGIN
44         CASE present_state IS
45             -- one case branch required for each state
46             WHEN idle =>
47                 IF failed = '1' THEN
48                     next_state <= perm_lock;
49                 ELSE
50                     next_state <= idle;
51                 END IF;
52                 IF enter = '1' THEN
53                     next_state <= Ev_code1;
54                 ELSE
55                     next_state <= idle;
56                 END IF;
57             WHEN Ev_code1 =>
58                 IF code = code1 THEN
59                     next_state <= get_code2;
60                 ELSE
61                     next_state <= idle;
62                     -- Set err_event to '1' when code1 is not equal to code
63                     IF code1 /= code THEN
64                         next_state <= wrong_code;
65                     ELSE
66                         next_state <= ev_code1;
67                     END IF;
68                 END IF;
69             WHEN get_code2 =>
70                 IF enter = '1' THEN

```

```

71         next_state <= Ev_code2;
72     ELSE
73         next_state <= get_code2;
74     END IF;
75 WHEN Ev_code2 =>
76     IF code = code2 THEN
77         next_state <= Unlocked;
78     ELSE
79         next_state <= idle;
80         -- Set err_event to '1' when code2 is not equal to code
81         IF code2 /= code THEN
82             next_state <= wrong_code;
83         ELSE
84             next_state <= idle;
85         END IF;
86     END IF;
87 WHEN Unlocked =>
88     IF enter = '1' THEN
89         next_state <= idle;
90     ELSE
91         next_state <= Unlocked;
92     END IF;
93 WHEN wrong_code =>
94     IF failed = '1' THEN
95         next_state <= perm_lock;
96     ELSE
97         next_state <= idle;
98     END IF;
99 WHEN perm_lock =>
100    next_state <= perm_lock;
101    -- default branch
102 WHEN OTHERS =>
103    next_state <= idle;
104 END CASE;
105 END PROCESS;
106 -----
107 ----- outputs -----
108 outputs : PROCESS (present_state, code, code1, code2, enter)
109 BEGIN
110     CASE present_state IS
111         -- one case branch required for each state
112     WHEN Unlocked =>
113         IF enter = '1' THEN
114             lock <= '1';
115             lock0 <= '0';
116         ELSE
117             lock <= '0';
118             lock0 <= '1';
119             err_event <= '0';
120         END IF;
121     WHEN wrong_code =>
122         err_event <= '1';
123         lock <= '1';
124         lock0 <= '0';
125         -- default branch
126     WHEN OTHERS =>
127         lock <= '1';
128         lock0 <= '0';
129         err_event <= '0';
130     END CASE;
131 END PROCESS;
132 -----
133 END processes;

```